Specora — Self-Healing Software

Pitch Deck — April 2026

The Hook

Your code breaks. Your contracts don't.

Specora is a platform where software fixes its own blueprints.

Write declarative contracts. Generate a running application. When something breaks, an AI reads the blueprint, proposes a fix, and the system regenerates. The bug is gone — permanently.

The Problem

Enterprise software is a house of cards

$2.4 trillion is spent annually on enterprise software and IT services (Gartner, 2025).

A staggering amount of that spend goes to:

- **Configuration that breaks on upgrade** — ServiceNow, Salesforce, SAP customizations that shatter every release cycle
- **Tribal knowledge** — the one person who knows how the system actually works
- **Bug whack-a-mole** — fix it in code, it comes back next sprint
- **Documentation rot** — specs that describe the system as it was designed, not as it works

The root cause: **code is the source of truth.** Code is fragile, opaque, and doesn't heal itself.

The Insight

What if the source of truth wasn't code?

Traditional:

Code → (breaks)

↓

Human fixes code

↓

Same bug returns

Specora:

Contracts → Code → (breaks)

↑

Healer fixes contract ←┘

↓

Bug is impossible

Contracts are declarative YAML specifications. They define what the system does — data models, state machines, API endpoints, auth rules — in a format that's readable, versionable, and machine-verifiable.

Code is generated from contracts. Delete it. Regenerate it. It's disposable. The contracts are the product.

The Demo (3 minutes)

Watch it happen

Step 1: Write 8 YAML contracts describing a helpdesk (agents, customers, tickets, lifecycle)

Step 2: Run one command — Forge generates a complete production application:

- FastAPI backend with Postgres
- Repository pattern (swap databases via env var)
- Docker deployment (3 containers)

- TypeScript types
- Pytest test suite

Step 3: docker compose up — the helpdesk is live. Create tickets, assign agents, transition states.

Step 4: Delete all generated code. Regenerate. Data survives. App is back.

Step 5: A runtime error occurs. The Healer catches it automatically, reads the contract, proposes a one-line fix via Claude. You approve on Discord. The contract updates. Code regenerates. Bug is gone forever.

[This demo was performed live on April 7, 2026. It is not a mockup.]

How It Works

Five tiers, one closed loop

Tier	Name	What it does
1	**Forge**	Contracts → production code. Compiler pipeline with 6 generator targets.
2	**Factory**	Conversation → contracts. LLM-powered authoring — "describe your app in English."
3	**Healer**	Errors → contract fixes. Self-healing pipeline with tiered autonomy.
4	**Extractor**	Existing code → contracts. Adopt Specora without rewriting.
5	**Advisor**	Telemetry → contract evolution. The system gets smarter over time.

The loop: **contracts → code → runtime → errors → contract fixes → regeneration → better code.**

No other platform closes this loop.

The Self-Healing Loop (Detail)

Runtime error → permanent fix in 60 seconds

1. App throws exception

↓ (automatic — error middleware)

2. Healer receives error

↓

3. AI reads contract + error + stacktrace

↓

4. Proposes specification-level fix

↓

5. You approve (CLI, API, or Discord/Slack/Teams)

↓

6. Contract modified on disk

↓

7. Forge regenerates code

↓

8. Docker rebuilds app

↓

9. Bug is structurally impossible

Tier 1 (naming/format errors): auto-fixed, no human needed. **Tier 2** (structural errors): LLM proposes, human approves. **Tier 3** (runtime errors): LLM traces to contract, human approves.

The Extractor — Adoption Without Rewriting

"Point at your codebase. Get self-healing contracts."

specora extract ./my-existing-app --domain my_app

The Extractor scans Python and TypeScript codebases:

1. Classifies files (models, routes, pages, config)
2. LLM extracts entities, fields, relationships, state machines
3. Cross-references and detects workflows

4. You review each entity (accept/skip)
5. Emits valid Specora contracts

Result: Your existing app now has contracts. From this point forward, it's self-healing and regeneratable.

This is the adoption wedge. No rewrite. No migration project. 15 minutes.

Market

Every company that builds software is a customer

Primary target: Enterprise IT departments running ServiceNow, Salesforce, Jira — paying millions for customization that breaks on upgrade.

Expansion: Any backend application. The contract language is domain-agnostic.

Vertical	Use Case
ITSM	Replace ServiceNow customization with self-healing contracts
Healthcare	HIPAA-compliant app generation with audit trails
Fintech	Regulatory-compliant API generation
SaaS	Backend generation for any data-driven product
Agencies	Client apps generated and maintained from specs
Government	Verifiable, auditable software from declarative specs

TAM: The global low-code/no-code market is $32B in 2025, growing 25% annually (Gartner). Specora targets the segment that low-code can't reach — complex, custom, backend-heavy applications.

Business Model

Open source engine, commercial cloud

Free (Open Source — Apache 2.0)

- Forge compiler + all generators
- Factory, Healer, Extractor
- Contract language + meta-schemas
- Docker deployment
- LLM-native interface

Builds adoption, trust, and community. The contract language becomes a standard.

Specora Cloud (SaaS)

Tier	Price	Features
Pro	$49-99/mo	Hosted Healer, dashboard, webhook integrations, contract analytics
Team	$199-499/mo	Multi-user approval, RBAC, audit log, Git integration, shared registry
Enterprise	$2K-10K/mo	Advisor tier, custom generators, SLA, SSO, migration services

Contract Marketplace (future)

- Community-shared domain contracts (helpdesk, e-commerce, healthcare)
- Verified premium contracts ($50-200)

Competitive Landscape

Nobody closes the loop

	Code Gen	Self-Healing	Reverse Engineer	LLM-Native
Specora	✅	✅	✅	✅
Amplication	✅	✗	✗	✗
JHipster	✅	✗	✗	✗

	Code Gen	Self-Healing	Reverse Engineer	LLM-Native
OutSystems	☑	✕	✕	✕
Mendix	☑	✕	✕	✕
Kubernetes	✕	⚠ infra only	✕	✕
Copilot	✕	⚠ code patches	✕	☑

Key differentiators:

1. Fixes the specification, not the code (permanent)
2. Generated code is disposable (delete and regenerate)
3. Existing apps can adopt without rewriting (Extractor)
4. Self-corrects when a prior fix was wrong
5. Patent pending on the specification-level feedback loop

Traction & Proof

Built and proven in 48 hours

Metric	Value
Automated tests	132
Generator targets	6 (FastAPI, Postgres, TypeScript, Docker, tests, legacy)
Contract kinds	7 (Entity, Workflow, Route, Page, Mixin, Infra, Agent)
LLM providers	6 (Anthropic, OpenAI, xAI, Z.AI, Google, Ollama)
Healer pipeline stages	7
Documentation	5,000+ lines

Metric	Value
Live demo	App + Postgres + Healer in Docker, self-healed via Claude

Live demonstration performed April 7-8, 2026:

- Generated a helpdesk app from 8 YAML contracts
- Deployed to Docker with Postgres
- Deleted all code, regenerated — data survived
- Runtime error auto-reported to Healer
- Claude proposed contract fix, approved via API
- Bug became structurally impossible after regeneration
- Healer self-corrected when first fix was too aggressive
- Discord notifications working in real time

IP & Defensibility

Patent pending + open source moat

Patent brief filed covering:

- Specification-level self-healing feedback loop
- Tiered autonomy for fix approval
- Runtime error to contract modification via LLM
- Sidecar deployment pattern for self-healing agent
- Reverse-engineering existing code into specifications
- Self-correction capability (Healer revises its own prior fixes)

13 claims drafted, attorney review pending.

Open source defensibility: Once the contract language is adopted, switching costs are real. Contracts are portable (YAML) but the ecosystem (generators, Healer, Advisor, marketplace) is not.

Roadmap

From engine to platform

Q2 2026 — Launch

- Open source release on GitHub
- "I deleted my code" blog post + demo video
- Target: 1,000 GitHub stars

Q3 2026 — Specora Cloud Beta

- Hosted Healer + dashboard
- Webhook integrations (Discord, Slack, Teams, Telegram approval bots)
- Target: 100 beta users, 10 paying ($5K MRR)

Q4 2026 — Team Tier

- Multi-user approval workflows
- Git integration (auto-PR for contract changes)
- Contract registry
- Target: $30K MRR

2027 — Enterprise

- Advisor tier (telemetry → proactive evolution)
- Custom generators
- Migration services
- Contract marketplace
- Target: $100K MRR

The Team

Nicholas Blanchard — Founder

- Background in ServiceNow/ITSM — saw the pain firsthand
- Built the predecessor (snow_cli, cdd-cmdb) proving the CDD concept

- Built the complete Specora Core engine in a single development session
- Solo technical founder (seeking co-founder for go-to-market)

The Ask

What we're looking for

If fundraising:

- Raising $750,000 pre-seed/seed or $150,000 - $200,000 SAFE note.
- 18-month runway to reach $100K MRR
- Use of funds: 1 full-time engineer, cloud infrastructure, go-to-market

If bootstrapping:

- First 10 design partners (companies willing to try the Extractor on their codebase)
- Introductions to enterprise IT leaders frustrated with ServiceNow/Salesforce customization
- Developer community amplification (blog posts, conference talks, podcasts)

Slide 16: The Close

One sentence

Specora is the first software platform that fixes its own blueprints — making bugs structurally impossible through declarative contracts, AI-powered healing, and code regeneration.

The engine is built. The patent is drafted. The demo runs on port 9000.

We're looking for the first believers.

Contact: cheapseatsecon@gmail.com

GitHub: github.com/syndicalt

Demo: available on request